Highlights

- Open-source Python framework for forest estate decision support.
- Model I scheduling with transparent inputs and outputs.
- libCBM linkage for carbon stock and flux accounting.
- Hybrid aspatial-to-raster workflows with reproducible geospatial I/O.
- Deterministic reproduction package and archived scaling benchmarks.

Software availability

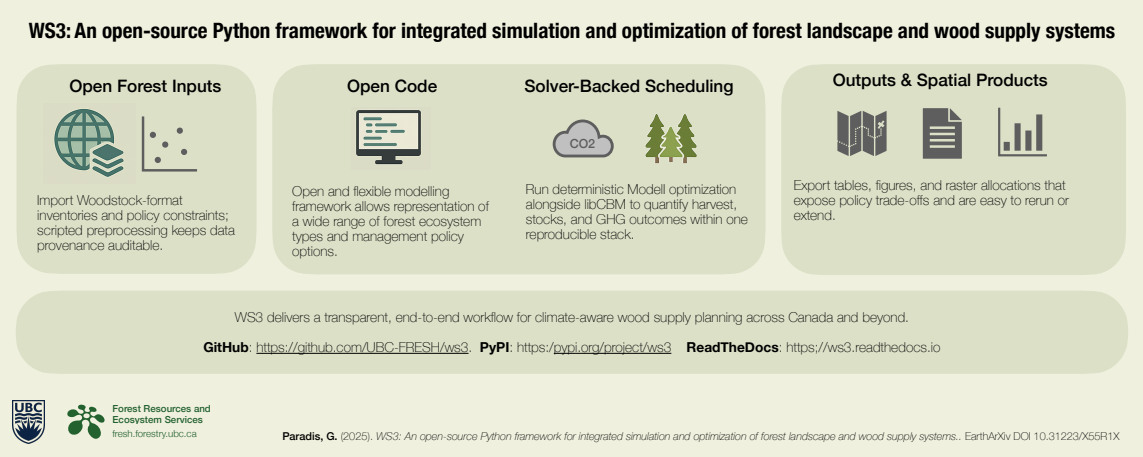
- Software name: WS3 (Wood Supply Simulation System)
- Developer and contact: Gregory Paradis (Faculty of Forestry, University of British Columbia); gregory.paradis@ubc.ca
- First public release and current version: 2015 (v0.1); current release v1.0.5
- Programming language and dependencies: Python (>=3.9); optional extras include `libcbm_py` (Carbon Budget Model Python bindings), PuLP (Python LP modeling) with the HiGHS solver, and Gurobi
- Source repository and issue tracker: `https://github.com/UBC-FRESH/ws3`
- Distribution channel: PyPI package `ws3` (`pip install ws3` or `ws3[cbm]`) with wheels for Linux, macOS, and Windows
- Documentation: `https://ws3.readthedocs.io`
- Archived release: Zenodo DOI 10.5281/zenodo.17331213
- License: MIT
- Hardware and OS requirements: Linux, macOS, or Windows; tested on Ubuntu 24.04 and macOS 14; CPU-only with recommended 16+ GB RAM for large LP instances
- Tested Python versions: 3.9–3.12 (continuous integration matrix)

- Data form and access: supplementary CSV, GeoTIFF, and notebook artifacts packaged in the GitHub repository and Zenodo release; total archive size under 50 MB excluding optional outputs

Graphical Abstract

WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis



WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis[a,*]

[a] *Faculty of Forestry, University of British Columbia, Canada*

Abstract

WS3 is an open-source Python framework that integrates Model I scheduling, carbon accounting, and raster allocation into a scriptable workflow for reproducible forest planning. It ingests Woodstock-format inventories, actions, and scenarios; defines an explicit data model; and automates aspatial–spatial conversion so harvest schedules can be mapped to disturbance footprints. A documented linkage to the Canadian Forest Service Carbon Budget Model (libCBM) provides carbon stock and flux estimates for libCBM-calibrated jurisdictions.

We document a modular architecture that decouples inventory ingestion, linear programming (LP) scheduling, carbon accounting, and raster allocation. Users can combine heuristic and LP approaches and extend indicators. We provide a reproducible parity check against Woodstock on a toy dataset and scaling experiments that characterize runtime and memory envelopes from desktop to cluster deployments. The complete toolchain—source code, configuration files, and deterministic reproduction scripts—is released under the MIT license with an archived Zenodo DOI. We demonstrate WS3 on five timber supply areas in northeastern British Columbia (20.40 Mha combined); post hoc libCBM runs provide carbon trajectories and raster allocation produces disturbance footprints for downstream spatial simulators. The framework is intended as reusable informatics infrastructure for decision support under competing sustainability mandates.

Keywords: computational ecology, decision support, forest planning, optimization, carbon accounting, geospatial workflows, open-source software

*Corresponding author: gregory.paradis@ubc.ca

1. Introduction

Forested landscapes sit at the center of intertwined climate mitigation, biodiversity, and bioeconomy mandates. Planners are being asked to reconcile wood supply security with measurable reductions in greenhouse-gas emissions across multi-decadal horizons [3, 15, 11]. Meeting these expectations demands decision-support systems that span harvest scheduling, regeneration and silviculture options, spatial footprint analysis, and explicit carbon accounting, while remaining transparent enough for regulators, Indigenous governments, and industrial partners to audit [27, 42]. This analytical load has outgrown the ad hoc spreadsheets and siloed software stacks that remain common in practice. It underscores the need for integrated modeling platforms that support reproducible workflows and open-science norms, including the PERFICT reproducibility principles for predictive ecology—a checklist for transparent, reusable computational workflows [20, 28, 29].

Strategic forest planning has long relied on operations research formulations such as Model I path scheduling and its variants, which underpin allowable-cut analyses and forest estate studies worldwide [23, 16, 17, 38]. Model I is a strata-based linear program in which decision variables assign fractions of aggregated development-type strata to prescriptions over time; this enables large-scale policy analysis but necessarily abstracts away stand identities. In practice, these ideas are delivered through proprietary toolchains—Woodstock, Patchworks, JLP (MELA), and Heureka, among others—that package data models, solvers, and reporting inside closed desktop environments [41, 43, 32, 21]. These systems are mature and widely trusted, yet closed licensing and opaque configuration formats can limit transparency, collaborative extension, and reproducible research workflows increasingly required by regulators and journals [20].

Open-source platforms have begun to address parts of this gap. Spatial simulation systems such as SpaDES and LANDIS-II, scenario-management frameworks like SyncroSim, and planning prototypes such as PRISM provide modular, shareable tooling [8, 26, 1, 34]. These systems excel at spatial dynamics and scenario orchestration but typically require users to supply planning modules, solver integrations, and carbon accounting pipelines to assemble a full decision-support stack [9, 27]. Parallel efforts such as the French CAPSIS platform highlight the promise and complexity of interoperable forest growth modeling ecosystems [13, 12, 40]. This gap between open simulation scaffolds and end-to-end forest estate planning motivates the framework described here.

WS3 is an open Python framework that implements legacy Model I scheduling while emphasizing reproducible, scriptable workflows with carbon accounting. It imports Woodstock-format

text files (or equivalent information in *ad hoc* formats via custom Python parsers) to protect historical investments, supports multiple LP solvers, couples to the Canadian Forest Service libCBM implementation, and supports hybrid spatial–aspatial workflows compatible with geospatial rasters and SpaDES-based simulations [41, 31, 19, 18, 25, 4, 8]. We do not claim a new optimization formulation; rather, we provide a transparent and auditable implementation that can be inspected, extended, and reproduced across institutions [20, 28].

This article contributes to ecological informatics in three ways. First, it documents a modular decision-support architecture that combines harvest scheduling, carbon accounting, and spatial reporting in a single, scriptable workflow. Second, it provides evidence of functional parity with Woodstock on a toy dataset and reports computational scaling experiments that characterize runtime and memory envelopes for large landscapes. Third, it makes the complete toolchain—source code, configuration files, and reproduction scripts—available under an open license with an archived Zenodo DOI, enabling immediate reuse and extension by researchers and practitioners. These contributions are workflow- and reproducibility-oriented; we do not propose a new optimization formulation.

The remainder of this article details the WS3 architecture and data model (Section 2), demonstrates reproducible use-case workflows linking LP-based scheduling to libCBM carbon accounting and spatial disturbance allocation (Section 3), discusses scope and limitations (Section 4), compares WS3 with alternative tools (Section 5), and summarizes availability, authorship, and future work to facilitate uptake.

2. Software description

2.1. Architecture

WS3 is organized into five roles: (i) core data abstractions; (ii) forest model construction and simulation; (iii) optimization; (iv) spatial allocation; and (v) common utilities. The main modules are:

- `forest`: defines the ForestModel and core abstractions for development types (dtypes), actions, transitions, yields, and scenario compilation. A scenario is specified by a planning horizon, period length, an initial inventory (areas by dtype and age), and transition rules governing state changes under actions and growth.

- `opt`: provides a linear-programming (LP) interface to formulate harvest-scheduling problems (objective, variables, constraints) and to solve them via PuLP with HiGHS by default; optional Gurobi.

- `spatial`: maps aspatial schedules to rasters (e.g., GeoTIFF) for visualization and spatial policy evaluation in a hybrid spatial/aspatial workflow.

- `core`, `common`, `financial`, `forest_helper`: shared utilities for interpolation and curves, example helpers, and optional financial indicators.

Data model and workflow. The ForestModel class uses discrete time (planning periods) and discrete age classes per dtype. Dtypes are keyed by tuples of theme values (e.g., analysis unit, leading species, site class). For optimization, WS3 aggregates stands into strata defined by dtype and optional zone masks (e.g., management units or policy zones). Spatial units (stands, polygons, or raster cells) appear only in the allocation step. Yields are represented by curves or interpolators. Actions (e.g., harvest, planting, fuel treatment, fire) encode operability and transitions (age reset, development type change, growth updates). WS3 builds a state-transition tree per development type across periods. Scheduling can be heuristic (area-control selectors that meet target areas by mask in priority order, e.g., oldest-first) or solver-based via a generic Model I linear program. Indicators are compiled as:

- Action-dependent flows via `compile_product(period, expr, acode=...)` (e.g., harvested volume using a utilization-adjusted expression on total volume yield).

- Inventory stocks via `inventory(period, yname=...)` (e.g., growing stock).

Interoperability. WS3 imports Woodstock-format text sections (LANDSCAPE, AREAS, YIELDS, ACTIONS, TRANSITIONS) so existing pipelines can be reused without re-encoding model logic. For carbon, ForestModel provides `to_cbm_sit` to export Standard Input Table (SIT) configuration and tables consumable by libCBM; users supply disturbance-type mappings and per-dtype last-pass-disturbance metadata. libCBM ships with parameterizations for multiple countries and ecozones [25], so the same export pathway supports analyses beyond Canada when users supply the appropriate classifiers and disturbance libraries. Geospatial I/O uses standard formats (shapefile, GeoTIFF). All tabular inputs are plain text (CSV/TSV), and workflows are scripted or notebook-based for reproducibility.

Figure 1 summarizes the WS3 workflow from open inputs through scheduling, carbon analysis, and spatial reporting. The end-to-end case study pipeline is implemented in the reproduction package and described below.

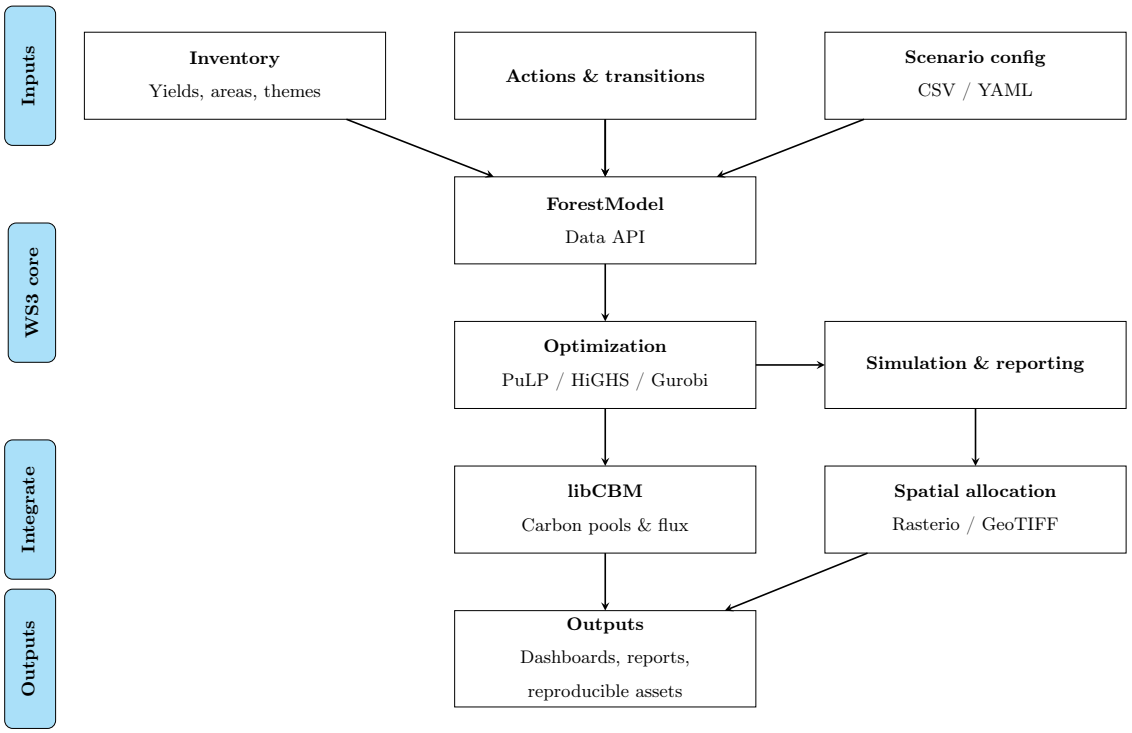


Figure 1: WS3 workflow: categorized inputs feed core scheduling and reporting modules, which integrate with libCBM and spatial allocation utilities before producing reproducible outputs.

The case study workflow implemented in the reproduction package loads Woodstock-format inputs, constructs the `ForestModel` instance, computes a schedule (heuristic or LP-based), exports a libCBM Standard Input Table formatted dataset, runs libCBM, allocates the aspatial schedule to raster space, and generates the figures and tables. This scripted pipeline underpins the sequential case study documented in Section 3. Note that WS3 is flexible and has been used in many workflows; the intent of the case study workflow shown here is illustrative rather than prescriptive or restrictive.

We summarize four practical contributions:

(i) A solver-flexible, path-based Model I generator embedded within a documented forest estate API, with parallel coefficient compilation for large instances.

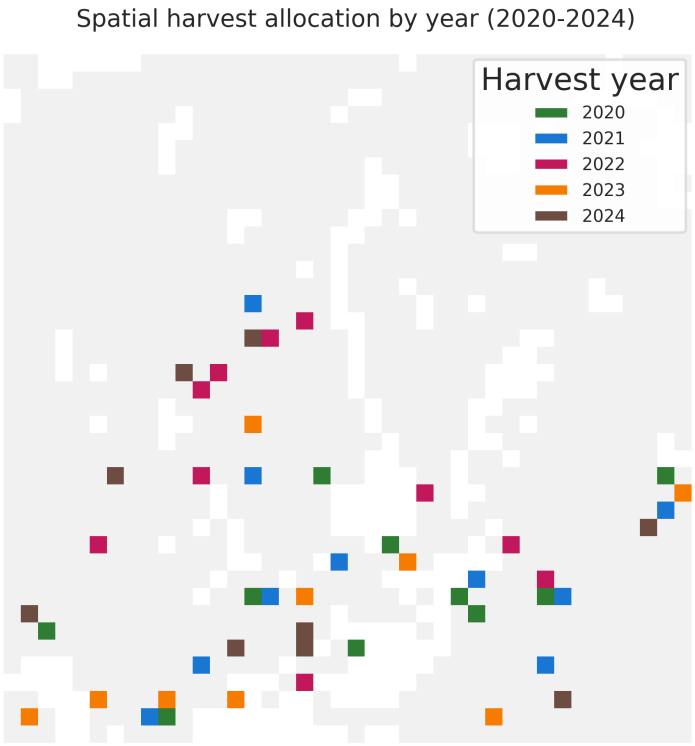


Figure 2: Heuristic schedule allocated to space for the 2020–2024 planning window. Colours denote the first year a cell is harvested, overlaid on the full inventory extent (light grey).

(ii) A documented libCBM interface that exports Standard Input Tables and ingests carbon pools and fluxes for integrated accounting.

(iii) A hybrid aspatial-to-raster allocation workflow that emits reproducible geospatial artifacts from deterministic schedules.

(iv) A FAIR-aligned reproduction package, including scaling benchmarks on real inventories and archived releases with pinned environments.

2.2. Optimization formulation (Model I)

WS3 adopts a classic Model I linear programming formulation [23, 16] in which each decision variable selects the proportion of a stratum (a dtype aggregate, optionally intersected with a zone

mask) allocated to a feasible root-to-leaf prescription (path) across the planning horizon. Stand identities are aggregated into strata for optimization, and spatial allocation is handled separately. Let:

I : set of strata (development types; aggregated zones)

J_i : set of feasible prescriptions (paths) for stratum $i \in I$

O : set of outputs (e.g., harvest area, harvest volume, growing stock, habitat)

$O' \subseteq O$: targeted outputs subject to even-flow constraints

T : set of planning periods

$T'_p \subseteq T$: periods on which even-flow for output $p \in O'$ is enforced

Decision variables are proportions:

$$x_{ij} \in [0,1] \quad \text{for all } i \in I,\ j \in J_i,$$

representing the fraction of stratum i assigned to prescription j. If A_i denotes the total area in stratum i, then $A_i x_{ij}$ is the area scheduled on path j. Let μ_{ijot} denote the quantity of output $o \in O$ in period $t \in T$ produced by allocating x_{ij} to path $j \in J_i$ for stratum $i \in I$. Define the reference-period output level $y_p := \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt_p^R} x_{ij}$ for each targeted output $p \in O'$ at $t_p^R \in T$, and ε_p as the allowable even-flow tolerance. With general lower/upper bounds v_{ot}^-, v_{ot}^+ on outputs, the Model I LP is:

$$\text{maximize} \quad \sum_{i \in I} \sum_{j \in J_i} c_{ij}\, x_{ij} \tag{1}$$

$$\text{subject to} \quad (1-\varepsilon_p)\, y_p \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt}\, x_{ij} \le (1+\varepsilon_p)\, y_p, \qquad \forall p \in O',\ \forall t \in T'_p \tag{2}$$

$$v_{ot}^- \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijot}\, x_{ij} \le v_{ot}^+, \qquad \forall o \in O,\ \forall t \in T \tag{3}$$

$$\sum_{j \in J_i} x_{ij} = 1, \qquad \forall i \in I \tag{4}$$

$$0 \le x_{ij} \le 1, \qquad \forall i \in I,\ \forall j \in J_i. \tag{5}$$

The objective coefficients c_{ij} encode the user-defined value of a path (e.g., total or discounted volume, revenues, multi-criteria scores). Constraints (2) enforce even-flow on targeted outputs, (3) set general lower/upper bounds per period, (4) ensures convex mixing of paths to fully cover each

stratum, and (5) bounds variables as proportions. In WS3, μ-coefficients arise from replaying each path through the simulator, calling `compile_product` (action-dependent outputs such as harvest area and volume) or `inventory` (stocks such as growing stock). The LP matrix is sparse because each path contributes to a limited set of outputs and periods.

Code-to-math mapping. WS3 compiles objective and constraint rows from paths using coefficient functions. Helper routines (e.g., `cmp_c_z`, `cmp_c_caa`, `cmp_c_ci`) traverse path nodes (period, action code, dtype key, age), evaluate μ-like contributions via `compile_product`/`inventory`, and assemble the LP with `fm.add_problem(...)`. Users select a solver (HiGHS by default; Gurobi optional) and solve via the standard PuLP interface. Upon optimality, WS3 applies the schedule and re-simulates to produce indicators.

Design implications. This path-based Model I approach embeds intertemporal feasibility in each column, separates simulation from optimization, and enables straightforward addition of new outputs/constraints by supplying new coefficient functions. It also aligns with established forest-planning literature while remaining solver-flexible and highly sparse for scalability.

2.3. Implementation

Technology stack. WS3 is implemented in Python and distributed via PyPI. Optimization problems are formulated by WS3 and solved with the open-source HiGHS [19] solver by default through the PuLP [31] Python LP interface, with optional Gurobi [18] bindings. Geospatial I/O uses rasterio, fiona, and GeoPandas. Documentation (Sphinx) and continuous integration (CI) pipelines provide API checks, runnable examples, and unit tests.

libCBM integration. The ForestModel class provides a `to_cbm_sit` method that compiles a CBM Standard Input Table (SIT) configuration and table data structure (i.e., classifiers, inventory, yields, disturbance events, transitions) consumable by libCBM. Users provide a disturbance type mapping and last-pass disturbance metadata to ensure correct dead organic matter (DOM) spin-up. The sequential demonstration runs libCBM for 200 years using the official Python API and documentation [6, 5].

Reproducibility and packaging. The repository includes examples and a reproduction package (`papers/ems/repro`) with pinned requirements and scripts to generate all figures and tables. Versioned releases are archived on Zenodo [35]. WS3 supports interactive (Jupyter notebooks) and batch (Python scripts) workflows and can be embedded in other systems as a standard Python

dependency.

2.4. Quality assurance and benchmarking

Automated testing. WS3 ships with a pytest suite that exercises the common, core, financial, forest, and optimization modules (29 tests). The suite runs in about 4 s on a Linux workstation (Python 3.12) and is executed for every push and pull request via a GitHub Actions workflow (Ubuntu, Python 3.10) that also regenerates a coverage badge [44]. Style tooling (Black, Ruff) is bundled in `requirements-dev.txt` and wired into the development Makefile, ensuring consistent formatting before release. Beyond unit tests, WS3 has been exercised in multiple operational planning projects over the past decade, providing repeated end-to-end validation of core workflows.

Input stewardship and validity. WS3, like its Woodstock heritage, makes no assumptions about inventories, yields, operability, objectives, or constraints. The space of "valid input" is project- and policy-specific and inherently high-dimensional; exhaustive auto-validation is out of scope. WS3 provides targeted checks in high-leverage paths (e.g., fallback to template transitions when age-specific rules are missing; basic range/shape sanity checks) and documentation of required structures for inventories, yields, actions, and transitions. Documentation is versioned with releases and focuses on core data structures and examples rather than serving as an exhaustive prescriptive manual. Responsibility for input quality and modeling assumptions rests with qualified analysts; WS3 is a professional framework rather than a prescriptive application.

Determinism and reproducibility. Given identical inputs, WS3 produces identical outputs. Minor nondeterminism is limited to optional solver seeds and randomized block ordering in the spatial allocation utility; both are controllable and fixed in the reproduction scripts. Consequently, statistical variance on deterministic time series is not meaningful; we report solved status, model scale, and runtime/throughput metrics instead. The reproduction package pins versions and seeds to yield byte-identical artifacts across runs, reflecting best practices for computational reproducibility [39].

Verification against Woodstock. WS3 was originally verified against Woodstock (2015) by replaying identical datasets and comparing action-specific flows and stocks across periods, yielding functionally equivalent outputs for the implemented subset of features. Known departures from Woodstock semantics are flagged in code and documentation. Ongoing development follows a trust-but-verify pattern: changes to core loops are regression-tested against known-good benchmarks

before release. The Woodstock parity check included in this manuscript is therefore a transparent, reproducible illustration rather than an exhaustive proof of equivalence.

Case study repro checks. The reproduction package encapsulates the sequential WS3→libCBM pipeline in `make_repro.sh`. In the project's reference LXD (Linux Containers) environment (Ubuntu 24.04 on dual Xeon 6254 CPUs, 72 logical cores, 768 GB PC-23400 RAM) the script completes in 6.1 s, recreating Figures 6 and 7, Figure 2, and the summary tables. The workflow is deterministic: running `generate_case_study.py` a second time produces byte-identical CSV and PNG assets, providing an integration test for the scheduling, SIT export, and libCBM coupling.

Performance spot checks. The heuristic scheduler used in the case study produces a 100-year plan in under a second, and the subsequent 200-year libCBM simulation dominates total runtime (~5 s). Larger linear programs formulated through `ws3.opt` benefit from HiGHS (default) or optional Gurobi: internal regression notebooks (e.g., Examples 030/040) solve tens of thousands of stand-path instances within minutes using HiGHS, and the optional Gurobi backend can reduce solve time further when available. Raster allocations scale linearly with the number of cells processed per period and can be parallelized by splitting periods or tiles.

Optional scalability experiment. To characterize scaling on real inventories, the reproduction package optionally integrates a DataLad-hosted benchmark dataset comprising five non-overlapping timber supply areas (TSAs) in northeastern British Columbia (the same study region as Boisvenue et al. 2). Enabling the optional step (`RUN_SCALING=1`) fetches the dataset and executes `papers/ems/repro/run_scaling_benchmarks.py`, which sorts TSAs by complexity and evaluates them individually and cumulatively (`tsa08`, `tsa08+tsa40`, ..., `tsa08+tsa40+tsa41+tsa24+tsa16`). The scripted benchmark always runs the heuristic scheduler single-threaded—reflecting the implementation's serial design—and records serial spatial-allocation runtimes using `ForestRaster`. On the reference container the heuristic schedules complete in 1.3–31.4 s while spatial allocation requires 46–656 s as problem size grows from 4.7 k to 37.7 k dtype–age pairs.

Setting `RUN_LP=1` extends the run to model-building benchmarks for the deterministic Model I LP. WS3 constructs the LP with 1 and 16 workers (parallel coefficient compilation) and solves it with matching HiGHS thread counts, logging build/solve time, solver status, and stage-wise memory footprints. LP builds scale from 5.1 s (4.7 k dtype–age pairs) to 92.8 s (37.7 k dtype–age pairs) with single-worker compilation, while the 16-worker mode trims build time to 59 s at the cost of higher peak resident set size (RSS; 2.5 GB versus 1.8 GB). Because large path-based LPs

routinely exceed tens of gigabytes of RAM on even larger inventories, the LP measurements are opt-in and intended for suitably provisioned systems. All metrics are written to `perf_scaling.csv` with deterministic seeds so that regenerated figures and tables remain byte-identical; Figures 3–5 and Table 1 summarize the resulting runtime and memory profiles.

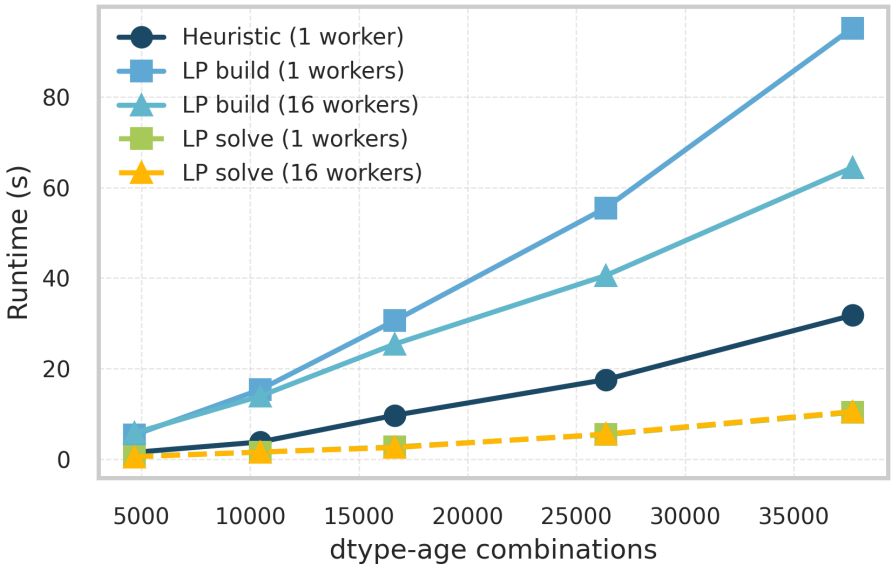


Figure 3: Scheduling runtime versus problem size, measured by the number of development type (dtype) by age combinations ($n_{\text{dtype-age}}$). Heuristic runs are single-worker; LP build/solve lines appear when `RUN_LP=1`.

3. Illustrative use-case demonstrations

We present a two-stage sequential pipeline that schedules harvest in WS3 and then evaluates carbon dynamics using libCBM. The workflow reproduces Example 031 (`031_ws3_libcbm_sequential-builtin.ipynb`) and is included to ensure reproducibility.

Study design. We use the public example dataset `tsa24_clipped`. It is intentionally modest so that the full pipeline (scheduling, carbon accounting, and spatial allocation) can be executed and audited on standard hardware, and it mirrors the tutorial dataset distributed with WS3 to keep inputs transparent and reproducible. Inputs are stored as Woodstock-format text files (landscape, areas, yields, actions, transitions) under `examples/data/woodstock_model_files_tsa24_clipped`

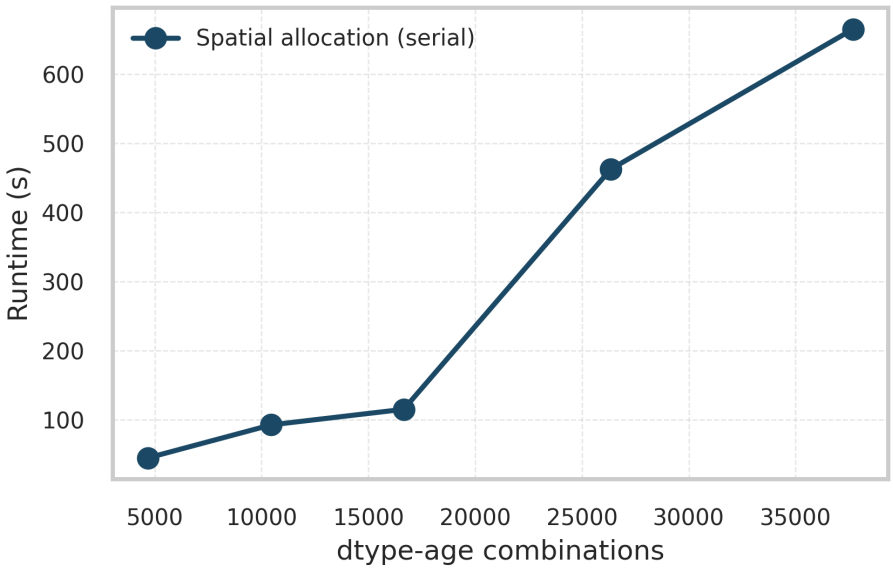


Figure 4: Spatial allocation runtime (serial `ForestRaster`) versus problem size after applying the heuristic schedule; problem size counts dtype–age combinations ($n_{\text{dtype-age}}$).

and imported into a `ForestModel`. We schedule harvest using a self-parameterizing area-control heuristic to produce an aspatial action schedule over a 100-year horizon (10 periods), then export a libCBM Standard Input Table (SIT) to simulate annual carbon stocks and fluxes.

Data and study area. The `tsa24_clipped` dataset is a clipped subset of a Timber Supply Area inventory used for demonstration. The initial inventory enumerates development types (dtypes) by theme tuples (e.g., analysis unit, species, site), with yield curves for total volume (`totvol`) and species-volume splits (`swdvol`, `hwdvol`). Actions and transitions define operability and state changes (e.g., harvest resets age).

Model setup and scheduling. We initialize the model with `base_year`=2020, `horizon`=10, `period_length`=10 y, and `max_age`=1000. After importing sections and initializing areas, we add a null action and reset actions. The area-control heuristic proceeds period by period: for each target mask (default: analysis-unit timber-harvesting land base (THLB)), it computes a target area using the area-weighted mean culmination of mean annual increment (CMAI) age and then applies the harvest action to the oldest operable ages within the masked dtype set until the target

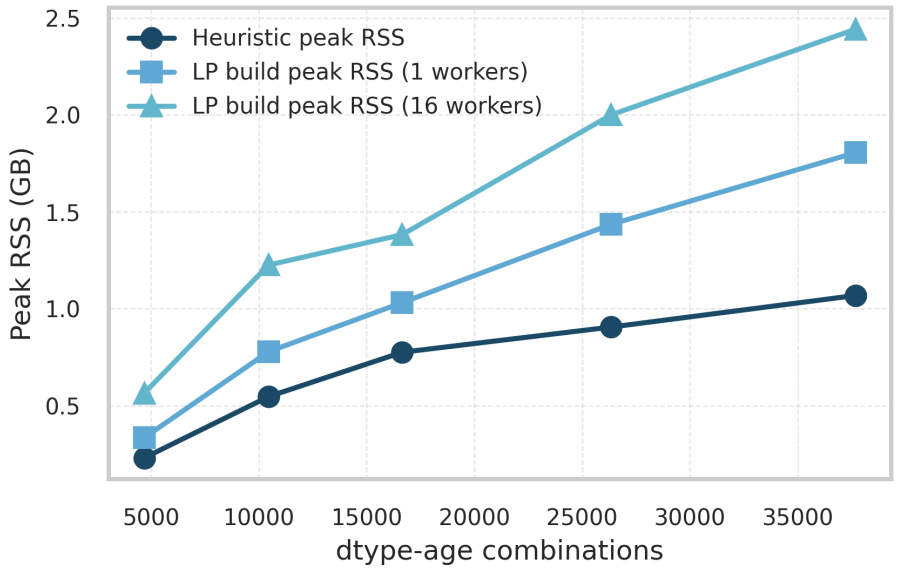


Figure 5: Peak resident memory for heuristic scheduling (serial) and LP model builds (1 and 16 workers) as problem size increases in terms of dtype–age combinations.

is met (or inventory is exhausted). This allocation step follows the hierarchical strategic→tactical paradigm described for CRYSTAL-style spatial allocation [22] and later operationalized in the Woodstock–Stanley workflow [10]. In commercial deployments, the spatialization module is now marketed as the Spatial Optimizer (formerly STANLEY) [30]. The resulting schedule is aspatial, recording treatment area by dtype, age, and period. Utilization is set to 0.85 in the volume expression used for compiled flows. The schedule is compiled and applied to produce period-by-period flows and stocks.

libCBM coupling and metrics. We define a disturbance-type mapping for actions (harvest→Clearcut harvesting without salvage; fire→Wildfire) and assign `last_pass_disturbance` per dtype to ensure consistent DOM spin-up. Using `ForestModel.to_cbm_sit` with softwood and hardwood volume names (`swdvol`, `hwdvol`), `admin_boundary` = `"British Columbia"`, and `eco_boundary` = `"Montane Cordillera"`, we export the SIT configuration and tables. We then run libCBM for 200 years (`n_steps`=200), aggregating annual carbon stocks (biomass, DOM, total ecosystem).

Outputs. We report time series of harvested area/volume and growing stock from WS3, and

Table 1: Peak resident memory during heuristic scheduling and LP model builds on the reference container. $n_{\text{dtype-age}}$ counts development type (dtype) by age combinations; LP measurements report the maximum RSS observed while compiling the Model I matrix with 1 or 16 workers, while spatial allocation runs serially.

Combo	$n_{\text{dtype-age}}$	Heuristic peak RSS (GB)	LP peak RSS (GB; 1 worker)	LP peak RSS (GB; 16 workers)
`tsa08`	4 682	0.23	0.33	0.56
`tsa08+tsa40`	10 453	0.52	0.78	1.22
`tsa08+tsa40+tsa41`	16 653	0.77	1.03	1.33
`tsa08+tsa40+tsa41+tsa24`	26 346	0.87	1.38	1.87
`tsa08+tsa40+tsa41+tsa24+tsa16`	37 691	1.07	1.81	2.51

annual carbon stocks from libCBM. Spatial allocation then maps the aspatial schedule onto raster cells using the ForestRaster utility to generate a disturbance footprint while preserving the aspatial totals. The reproduction package generates the figures and tables used below.

The full, deterministic workflow is scripted in `papers/ems/repro/generate_case_study.py`, which produces the flows and carbon stock tables:

- `papers/ems/tables/scenario_flows.csv`: harvest area, harvest volume, and growing stock by period.
- `papers/ems/tables/annual_carbon_stocks.csv`: annual biomass, DOM, and total ecosystem carbon.

Figures 6 and 7 show the resulting flows and carbon stocks. The entire pipeline, including the workflow overview (Figure 1) and the spatial allocation example (Figure 2), is reproducible via `papers/ems/repro/make_repro.sh`. The libCBM run length (200 years) matches the example but can be adjusted.

Unless stated otherwise, areas are reported in hectares (ha) and volumes in cubic metres (m^3).

Results. The heuristic scheduler delivers a 10-period harvest plan whose flows stay within ~10% of period means without any explicit even-flow constraint. Harvest area averages 106.8 ha and ranges from 103 to 114 ha, while harvest volume averages 15.3 thousand m^3 and ranges from 14.1–18.5 thousand m^3 (Table 2; Figure 6). Growing stock declines smoothly from 135 thousand to

Table 2: Sequential demonstration parameters and configuration (WS3 → libCBM pipeline).

Setting	Value
Dataset	`tsa24_clipped` (Woodstock-format inputs; `examples/data/woodstock_model_files_tsa24_clipped`)
Base year	2020
Planning horizon	10 periods (100 years)
Period length	10 years
Max age class	1000 years
Yield names	`totvol` (total volume), `swdvol` (softwood), `hwdvol` (hardwood)
Scheduling method	Area-control heuristic (priority-queue, oldest-first); utilization 0.85
Disturbance mapping (CBM)	harvest→Clearcut harvesting without salvage; fire→Wildfire
CBM export	`ForestModel.to_cbm_sit` (admin: British Columbia; eco: Montane Cordillera)
CBM run length	200 annual steps
Outputs	`scenario_flows.csv` (period, harvest_area_ha, harvest_volume_m3, growing_stock_m3); `annual_carbon_stocks.csv` (annual pools)

82 thousand m^3 (consistent with the 0.85 utilization factor), reflecting the state trajectory exported to libCBM. In the carbon stage, libCBM aggregates show biomass pools increasing from 2.7×10^5 tC to 3.1×10^5 tC over 200 years, while DOM remains within 1.7–1.9 $\times 10^5$ tC, yielding a total ecosystem carbon gain of 42 ktC (Figure 7). These values are illustrative; the key contribution is the deterministic, auditable pipeline that connects scheduling outputs to carbon accounting and spatial allocation with byte-identical outputs. The summary tables `scenario_flows.csv` and `annual_carbon_stocks.csv` (`papers/ems/tables`) retain raw outputs for external verification.

Reproducibility. This demonstration is intended to be transparent and portable. All input data reside in `examples/data/woodstock_model_files_tsa24_clipped`. The spatial allocation step

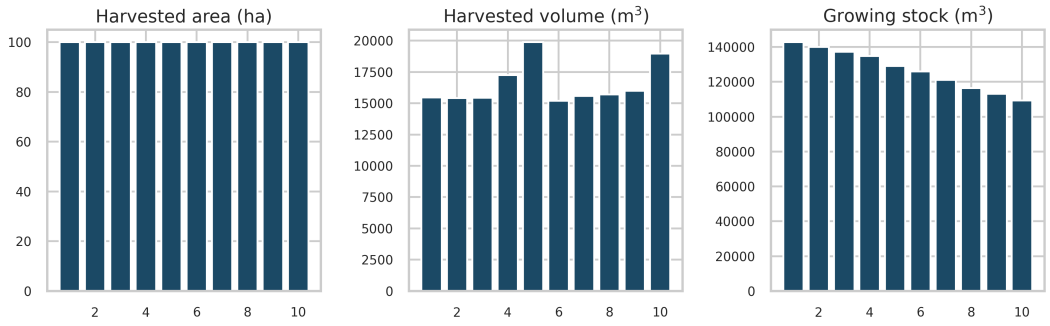


Figure 6: WS3 results: harvested area, harvested volume (utilization-adjusted), and growing stock over planning periods.

produces the raster-based harvest allocation shown in Figure 2, derived directly from the aspatial schedule via the ForestRaster utility. The folder `papers/ems/repro` contains `make_repro.sh`, which provisions a clean virtual environment, installs pinned dependencies (including the optional libCBM extra via `pip install ws3[cbm]`), and runs deterministic scripts that regenerate Figures 1–7 and the accompanying CSV tables. Outputs are byte-identical on the reference container and stable under the pinned dependencies. The exact environment is preserved in the Zenodo-archived release.

Broader example library and carbon-aware optimization

Beyond the sequential carbon accounting demonstration, WS3 includes executable notebooks that serve as tutorials and validation exercises, ranging from synthetic cases to policy-oriented analyses. Example 040 implements the Neilson hack and compares aggregated WS3 carbon indicators with libCBM in a no-disturbance scenario. Example 041 illustrates carbon-aware optimization by embedding carbon prices/constraints in the LP and evaluating schedules with `libcbm_py`.

4. Impact and reusability

WS3 has been used in research on bioenergy [7], value-creation potential and supply modeling [37, 36], and climate impact assessment [42, 24, 48]. It integrates with the SpaDES ecosystem for spatial simulation [8] and has been deployed as a backend in decision-support applications. The MIT license, modular API, and PyPI distribution support reuse across jurisdictions.

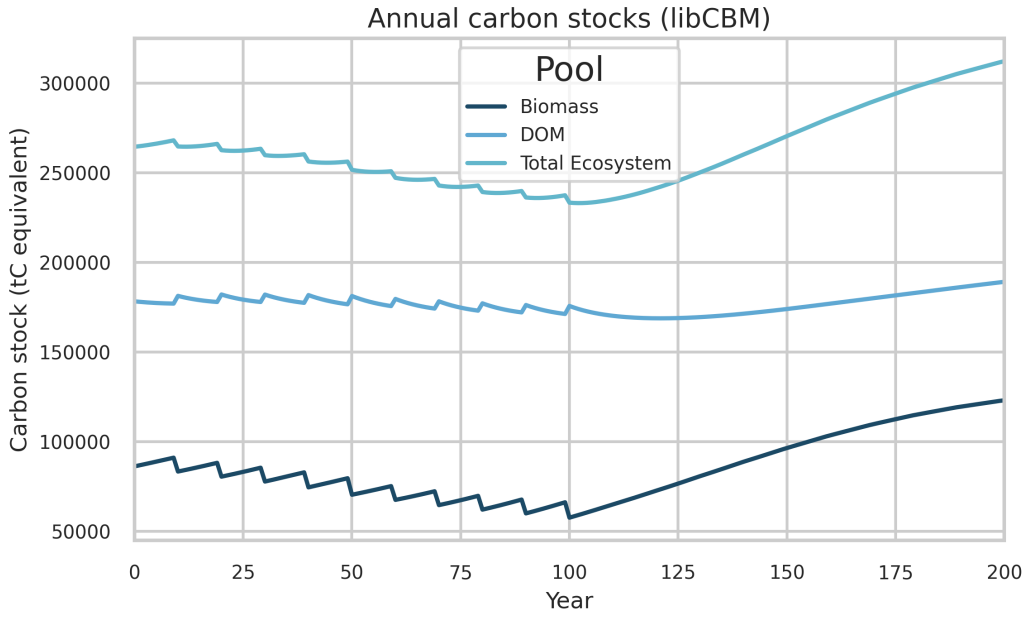


Figure 7: libCBM results: annual carbon stocks (biomass, dead organic matter, total ecosystem) over 200 years.

Major use cases (2015–2025).

- Strategic wood supply and bioenergy planning in mixed-wood forests: optimization and scenario analysis of value-creation options [7].
- Hybrid simulation–optimization for value indicators and model retrofitting: methods and transfer to production-style analyses [37, 36].
- Climate mitigation and carbon-aware planning in British Columbia: sequential WS3→libCBM pipelines and optimization under policy constraints; graduate theses and applied demonstrations [24, 48, 47].
- Decision-support prototypes for nature-based solutions: avoided fire and avoided harvest workflows (Examples 050/060) with net-emission indicators derived from libCBM.
- Integration with spatial simulation: `spades_ws3` module bridging WS3 with SpaDES for disturbance and landscape dynamics studies [8, 46].

- Application backends: WS3 used in web-based decision-support systems (e.g., `ecotrust-dss`) for scenario configuration and reporting [45].
- Ongoing initiatives: CCCANDiES (integrated carbon and ecosystem services; in progress) and mining-sector nature-based solutions [14].

Interoperability is supported through tabular/text inputs, raster outputs, and the libCBM linkage, enabling WS3 to integrate with existing analytical pipelines. The reproduction package, unit tests, and continuous integration (CI) pipelines facilitate method transfer and support open scientific practices [20, 28]. The architecture is general and has been applied to studies of sustainable yield, carbon accounting, biodiversity, and policy trade-offs [2].

Reusability features.

- Openness and packaging: MIT license, PyPI distribution, versioned releases archived on Zenodo [35]; complete examples and tutorials.
- Interoperability: Woodstock-format import for legacy/industry pipelines; libCBM SIT export for carbon; standard geospatial formats (GeoTIFF, shapefiles).
- Reproducibility: deterministic reproduction package (`papers/ems/repro`) that regenerates all figures and tables; continuous integration (CI) pipelines run tests and validate examples.
- Extensibility: modular API for adding outputs/constraints and new scheduling logic (heuristic or LP-based); solver-flexible (HiGHS by default; Gurobi optional).
- Integration pathways: SpaDES ecosystem via `spades_ws3`; backend for decision-support applications and web services.
- Education and onboarding: progressive example notebooks (from `010` to `060`) facilitate training and method transfer.

Example 040 demonstrates a stand-level optimization workflow with explicit integration of carbon dynamics using `libcbm_py`. The workflow shows how WS3's optimization layer and libCBM coupling can support carbon-aware planning experiments while leaving the precise objective formulation to the user.

Table 3: Comparison of WS3 with selected tools.

Tool	Scope	License	Optimization	Spatial	Carbon	Region
WS3	Landscape	Open (MIT)	Built-in (PuLP; optional Gurobi)	Hybrid aspatial-to-raster	Built-in libCBM linkage	General
Woodstock	Landscape	Proprietary	Built-in	Extensions; connectors	External integrations	General
Patchworks	Landscape	Proprietary	Built-in	Spatial planning emphasis	External integrations	General
SpaDES	Simulation framework	Open	External packages	Fully spatial (agent-based, raster)	Via modules	General
Heureka	Landscape	Restricted (mixed)	Built-in	Spatial capabilities	External integrations	EU-focused
JLP (MELA)	Landscape	Restricted (legacy)	Built-in	Spatial extensions	External integrations	Finland-focused

5. Comparison to alternatives

Parity with Woodstock. To demonstrate that WS3 implements standard inventory accounting correctly, we report a parity check against a reference Woodstock run using the same toy dataset and an identical action schedule. This test is a transparent, reproducible illustration rather than an exhaustive proof of equivalence. The accompanying CSV artifact `papers/ems/tables/woodstock_parity.csv` provides total harvest area, harvest volume, and growing stock, along with WS3-relative percent differences (computed as $100\,(\text{WS3}-\text{Woodstock})/\text{Woodstock}$). For transparency, period-wise parity summaries for harvest area and volume can be regenerated via the reproduction scripts and are included in the reproduction package (Figure 9). Exact or near-exact agreement is expected given identical inputs.

WS3 provides an open Python API, a documented libCBM integration, and a hybrid spatial/aspatial workflow intended for reproducible analyses. It is a framework rather than a GUI decision-support suite; spatial adjacency, opening-size constraints, and road-network planning are outside its scope. Woodstock, Patchworks, Heureka, and JLP target landscape-scale planning with varying degrees of spatial explicitness and availability [41, 43, 8, 21, 32].

Table 4: WS3 vs. Woodstock parity on the toy dataset (identical schedule). Values are loaded from the accompanying CSV artifact.

Metric	WS3	Woodstock	WS3 vs Woodstock (%)
Total harvest area (ha)	1,000.00	1,000.00	0.00
Total harvest volume (m^3)	164,838.08	164,838.08	0.00
Total growing stock (m^3)	1,269,692.15	1,269,692.15	0.00

Comparison narrative. Proprietary landscape systems such as Woodstock and Patchworks offer mature optimization and spatial planning capabilities, including native support for adjacency and block constraints, but their licensing and closed codebases can limit transparent methods development and reproducibility at scale. SpaDES is an open simulation framework oriented to spatial processes and agent-based dynamics; it does not natively provide a solver-backed harvest scheduling layer but can interoperate with WS3 via `spades_ws3` for hybrid workflows. Heureka and JLP are landscape-oriented and include optimization, yet are region-specific or legacy-maintained, which constrains adoption beyond their primary jurisdictions.

Carbon-aware planning and optimization have an active literature that couples timber supply models with carbon accounting and evaluates mitigation trade-offs under alternative schedules and policies. Early integrations such as CO2Fix paired with timber supply modeling illustrate carbon-informed planning workflows [33]. More recent sector and landscape studies quantify mitigation potential and carbon capacities under harvest scenarios [42, 2], and graduate theses in British Columbia explicitly evaluate climate objectives in forest planning optimization contexts [24, 48]. WS3 provides a reproducible, open framework for constructing these workflows while keeping the scheduling engine transparent and auditable.

WS3 integrates: (i) a solver-flexible, path-based Model I LP generator embedded in an open forest estate API; (ii) a documented linkage to libCBM for carbon stocks and fluxes; and (iii) a hybrid spatial/aspatial pipeline with standard geospatial I/O. This design supports reproducible workflows in notebooks and scripts and integrates with larger research and decision-support ecosystems. Where spatially explicit optimization is required, WS3 can provide schedules and carbon accounting inputs to specialized spatial tools; carbon-related objectives or constraints can be added when users supply coefficients, but they are not required for the core workflow demonstrated here.

6. Discussion

WS3 is intended as an open, reproducible alternative to proprietary forest estate planning systems. Its contribution is primarily informatics: a transparent, scriptable workflow that integrates scheduling, carbon accounting, and spatial reporting, rather than a new optimization formulation.

6.1. Scope and limitations

WS3 focuses on deterministic, aspatial forest estate formulations. It does not attempt to represent stochastic risk, uncertainty propagation, machine learning forecasting, or fully spatial optimization; those are handled through scenario ensembles or specialized tools rather than within the core framework. The Model I LP implementation is well suited to even-flow policies, policy bounds, and value maximization problems, yet it inherits classic limitations of the formulation: stochastic or endogenous disturbances cannot be represented faithfully without leaving linearity, and nonlinear objectives or constraints require either linearization or external wrappers. In practice, we use the heuristic scheduler to run disturbance realizations or sensitivity analyses when scenario logic demands it, and reserve the LP for static policy experiments.

Spatial representation is provided through a post hoc raster allocation step that respects operability and transitions but does not enforce spatial adjacency, road-building, or block-shape constraints. Those dynamics fall outside the scope of WS3. Users requiring spatial optimization should therefore treat WS3 as a scheduling and accounting engine that feeds specialized spatial tools.

In this manuscript, carbon accounting is applied post hoc to schedules produced by the LP or heuristic workflows. The framework can be extended to incorporate carbon-related objectives or constraints when users supply the necessary coefficients, but those formulations are not the focus of the present study.

Finally, large LP instances involve substantial memory pressure—tens of gigabytes for the largest benchmarks—because the column generation compiles full path matrices. The reproduction package flags the LP scaling experiment as opt-in so that teams can execute it on appropriately provisioned infrastructure. All supporting assets remain deterministic and auditable even when the LP portion is skipped.

7. Conclusions and future work

WS3 provides an open, reproducible framework for integrating harvest scheduling, simulation, carbon accounting, and spatial reporting. By coupling solver-backed scheduling with libCBM via a documented interface, WS3 supports transparent Model I analyses that can be inspected, reproduced, and extended.

Limitations remain: results depend on the quality of user-supplied yields, transition rules, and carbon parameters; stochastic disturbances and climate-forced growth uncertainty require external scenario ensembles; and large LP instances can require substantial memory. The reproduction package highlights these constraints and provides deterministic assets for audit and reuse.

Future work will prioritize uncertainty handling and spatial extensions, guided by community contributions to the open codebase.

Reproducibility. All figures and tables are generated by scripts in `papers/ems/repro`. The `make_repro.sh` script provisions an isolated environment, installs pinned dependencies (`requirements.txt`), and runs `generate_diagrams.py`, the spatial allocation workflow (`generate_spatial_allocation.py`), and the sequential demonstration script (`generate_case_study.py`). Outputs are written to `papers/ems/figs` and `papers/ems/tables`. The exact versions are preserved via the Zenodo-archived release.

FAIR compliance. Table 5 summarizes the FAIR/software checklist submitted with the manuscript (see supplementary CSV `fair_checklist.csv`). WS3 remains Findable, Accessible, Interoperable, and Reusable through its Zenodo DOI archive and indexed repositories, MIT-licensed code/data artifacts, open tabular and geospatial formats plus Woodstock/libCBM linkages, and continuous integration (CI)–tested workflows with full reproduction scripts [35].

CRediT authorship contribution statement

G.E.P.: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review & editing, Visualization, Supervision, Project administration, Funding acquisition.

Table 5: FAIR/software checklist summary for WS3.

Principle	Checklist focus	Evidence / artifact
Findable	Persistent identifier; indexed repository	Zenodo DOI 10.5281/zenodo.17331213; tagged GitHub releases; PyPI project metadata
Accessible	License and public access	MIT license (CITATION.cff, LICENSE); public GitHub repository; PyPI wheels; reproduction package assets bundled in repository
Interoperable	Open formats and connectors	Woodstock LAN/ARE/YLD importers; libCBM SIT export; tabular CSV, GeoTIFF, and shapefile outputs; documented workflow in Figure 1
Reusable	Documentation, provenance, validation	ReadTheDocs site; notebooks/examples; continuous integration (CI)–tested `pytest` suite; deterministic scripts in `papers/ems/repro`; submission checklist metadata

Acknowledgements

Development of WS3 was supported by the Forest Resources Environmental Services Hub (FRESH) at the University of British Columbia. The author thanks Elaheh Ghasemi, Salar Ghotb, and Kathleen Coupland for contributions to documentation, testing, and code quality improvements. Language polishing and compliance checks were assisted by the OpenAI GPT-5 and GPT-5-Codex models; the author reviewed and edited all outputs.

Funding

Natural Sciences and Engineering Research Council of Canada (NSERC, grant RGPIN-2023-04197); Environment and Climate Change Canada (ECCC, grant 3000770190); Natural Resources Canada (NRCan, grant 3000771054); Government of British Columbia (grant TP24FCCS004); Mitacs (Newmont Goldcorp Inc., grant IT32088).

Data availability

Source code, configuration files, and reproduction assets are archived at Zenodo (DOI: `10.5281/zenodo.17331213`). All scripts used to generate the figures and tables are included; reuse is

permitted under the MIT License.

Declaration of competing interest

The author declares no competing interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used the OpenAI GPT-5 and GPT-5-Codex models to assist with language polishing and format compliance. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] Apex Resource Management Solutions, 2024. SyncroSim: Scenario Modeling Platform [software]. URL: https://syncrosim.com. accessed 2025-09-28.

[2] Boisvenue, C., Paradis, G., Eddy, I.M., McIntire, E.J., Chubaty, A.M., 2022. Managing forest carbon and landscape capacities. Environmental Research Letters 17, 114013. doi:10.1088/1748-9326/ac9919.

[3] Canadell, J.G., Monteiro, P.M.S., Joos, F., et al., 2022. Agriculture, forestry and other land use, in: Shukla, P.R., Skea, J., Slade, R., et al. (Eds.), Climate Change 2022: Mitigation of Climate Change. Contribution of Working Group III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change. Cambridge University Press, Cambridge, UK, pp. 751–850. URL: https://keneamazon.net/Documents/Publications/Virtual-Library/Impacto/157.pdf.

[4] Canadian Forest Service, 2025a. libcbm-py: Python implementation of the Carbon Budget Model [software]. URL: https://pypi.org/project/libcbm/. Python package (accessed 2025-09-28).

[5] Canadian Forest Service, 2025b. libcbm_py: Carbon Budget Model Python Interface [software]. URL: https://github.com/cat-cfs/libcbm_py. source code repository (accessed 2025-09-28).

[6] Canadian Forest Service, 2025c. libcbm_py Documentation. URL: https://cat-cfs.github.io/libcbm_py/. documentation site. (accessed 2025-09-28).

[7] Cantegril, P., Paradis, G., LeBel, L., Raulier, F., 2019. Bioenergy production to improve value-creation potential of strategic forest management plans in mixed-wood forests of eastern canada. Applied Energy 247, 171–181. doi:10.1016/j.apenergy.2019.04.022.

[8] Chubaty, A.M., McIntire, E.J.B., et al., 2024. SpaDES: Spatially Explicit Discrete Event Simulation [software]. URL: https://spades.predictiveecology.org. r package and framework (accessed 2025-09-28).

[9] Daigneault, A.J., Hayes, D.J., Fernandez, I.J., Weiskittel, A.R., 2022. Forest carbon accounting and modeling framework alternatives: an inventory, assessment, and application guide for Eastern US State Policy Agencies. Technical Report. University of Maine, Center for Research on Sustainable Forests. URL: https://crsf.umaine.edu/resource/forest-carbon-accounting-and-modeling-guide/.

[10] Feunekes, U., Cogswell, A., 1997. A hierarchical approach to spatial forest planning. General Technical Report NC-205. USDA Forest Service, North Central Forest Experiment Station. St. Paul, MN, USA. URL: https://remsoft.com/wp-content/uploads/2020/06/Remsoft_SSAFR-1997_-Hierarchical-Approach-to-Spatial-Forest-Planning.pdf.

[11] Food and Agriculture Organization of the United Nations, 2020. Global Forest Resources Assessment 2020. Main Report. Technical Report. Food and Agriculture Organization of the United Nations. Rome. doi:10.4060/ca9825en.

[12] Fortin, M., Lavoie, J.F., 2022. Reconciling individual-based forest growth models with landscape-level studies through a meta-modelling approach. Canadian Journal of Forest Research 52, 1140–1153. doi:10.1139/cjfr-2022-0002.

[13] Fortin, M., Sattler, D., Schneider, R., 2022. An alternative simulation framework to evaluate

the sustainability of annual harvest on large forest estates. Canadian Journal of Forest Research 52, 704–715. doi:10.1139/cjfr-2021-0255.

[14] Ghasemi, E., Coupland, K., Ghotb, S., Paradis, G., 2025. Developing a prototype decision support framework to assess forest management scenarios as a nature-based decarbonization solution for the mining sector: A case study in british columbia, canada. EarthArXiv. URL: https://eartharxiv.org/repository/view/10522/, doi:10.31223/X5W731.

[15] Griscom, B.W., Adams, J., Ellis, P.W., et al., 2017. Natural climate solutions. Proceedings of the National Academy of Sciences 114, 11645–11650. doi:10.1073/pnas.1710465114.

[16] Gunn, E., 2007. Models for Strategic Forest Management, in: Weintraub, A., Romero, C., Bjørndal, T., Epstein, R. (Eds.), Handbook of Operations Research in Natural Resources. Springer Science+Business Media. chapter 16, pp. 317–341. doi:10.1007/978-0-387-71815-6_16.

[17] Gunn, E., 2009. Some perspectives on strategic forest management models and the forest products supply chain. INFOR: Information Systems and Operational Research 47, 261–272. doi:10.3138/infor.47.3.261.

[18] Gurobi Optimization, LLC, 2024. Gurobi Optimizer [software]. URL: https://www.gurobi.com. accessed 2025-09-28.

[19] Hall, J., et al., 2023. HiGHS: High performance software for linear optimization [software]. URL: https://highs.dev/. accessed 2025-09-28.

[20] Hampton, S.E., Anderson, S.S., Bagby, S.C., et al., 2015. The tao of open science for ecology. Ecosphere 6, 1–13. doi:10.1890/ES14-00402.1.

[21] Heureka Consortium, 2024. Heureka Forestry Decision Support System [software]. URL: https://www.heurekaslu.se/. accessed 2025-09-28.

[22] Jamnick, M.S., Walters, K.R., 1993. Spatial and temporal allocation of stratum-based harvest schedules. Canadian Journal of Forest Research 23, 402–413. doi:10.1139/x93-058.

[23] Johnson, K.N., Scheurman, H.L., 1977. Techniques for prescribing optimal timber harvest and investment under different objectives: discussion and synthesis. Forest Science 23. doi:10.1093/forestscience/23.s1.a0001. monograph 18.

[24] Ke, J., 2024. Climate impact assessment under different forest harvesting and fertilization scenarios. Master's thesis. University of British Columbia. doi:10.14288/1.0441338.

[25] Kurz, W., Dymond, C., Stinson, G., Rampley, G., et al., 2009. CBM-CFS3: A model of carbon-dynamics in forestry and land use change. Canadian Journal of Forest Research 39, 495–509. doi:10.1016/j.ecolmodel.2008.10.018.

[26] LANDIS-II Foundation, 2024. LANDIS-II Forest Landscape Model [software]. URL: https://www.landis-ii.org. accessed 2025-09-28.

[27] Law, B.E., Hudiburg, T.W., Berner, L.T., 2018. Land use strategies to mitigate climate change in carbon dense temperate forests. Proceedings of the National Academy of Sciences 115, 3663–3668. doi:10.1073/pnas.1720064115.

[28] Lowndes, J.S.S., Best, B.D., Scarborough, C., et al., 2017. Our path to better science in less time using open data science tools. Nature Ecology & Evolution 1, 0160. doi:10.1038/s41559-017-0160.

[29] McIntire, E.J., Chubaty, A.M., Cumming, S.G., Andison, D., Barros, C., Boisvenue, C., Haché, S., Luo, Y., Micheletti, T., Stewart, F.E., 2022. PERFICT: A Re-imagined foundation for predictive ecology. Ecology Letters 25, 1345–1351. doi:10.1111/ele.13994.

[30] Mistik Management Ltd., 2019. 2019 20-year forest management plan: Volume II Document V – Modeling assumptions. Technical Report. Mistik Management Ltd.. Meadow Lake, Saskatchewan, Canada. URL: https://www.mistik.ca/wp-content/uploads/2019-Documents/Vol_II_Doc5_Modeling_Assumptions.pdf.

[31] Mitchell, S., et al., 2023. PuLP: A Linear Programming Toolkit for Python [software]. URL: https://coin-or.github.io/pulp/. accessed 2025-09-28.

[32] Natural Resources Institute Finland (Luke), 2024. JLP (MELA) Forest Planning System. URL: http://mela2.metla.fi/mela/j/jlp-en.htm. accessed 2025-09-28.

[33] Neilson, E.T., MacLean, D.A., Arp, P.A., Meng, F.R., Bourque, C.P., Bhatti, J.S., 2006. Modeling carbon sequestration with CO2Fix and a timber supply model for use in forest management planning. Canadian Journal of Soil Science 86, 219–233. doi:10.4141/S05-081.

[34] Nguyen, D., Henderson, E., Wei, Y., 2022. Prism: A decision support system for forest planning. Environmental Modelling & Software 157, 105515. doi:10.1016/j.envsoft.2022.105515.

[35] Paradis, G., 2025. ws3: Wood Supply Simulation System [software]. doi:10.5281/zenodo.17331213.

[36] Paradis, G., Lebel, L., 2018a. Estimating the value-creation potential of wood supply using a hybrid simulation-optimization approach. Technical Report CIRRELT-2018-24. Centre interuniversitaire de recherche sur les réseaux d'entreprise, la logistique, et le transport (CIRRELT). doi:10.13140/RG.2.2.32706.48321.

[37] Paradis, G., Lebel, L., 2018b. Retro-Fitting Value-Creation Potential Indicators to Long-Term Supply Models. Technical Report CIRRELT-2018-23. Centre interuniversitaire de recherche sur les réseaux d'entreprise, la logistique, et le transport (CIRRELT). doi:10.13140/RG.2.2.19284.71040.

[38] Paradis, G., LeBel, L., D'Amours, S., Bouchard, M., 2013. On the risk of systematic drift under incoherent hierarchical forest management planning. Canadian Journal of Forest Research 43, 480–492. doi:10.1139/cjfr-2012-0334.

[39] Peng, R.D., 2011. Reproducible research in computational science. Science 334, 1226–1227. doi:10.1126/science.1213847.

[40] Power, H., 2015. Guide d'utilisation du simulateur de croissance forestière Artémis-2014 sur Capsis (Version préliminaire 3.1). Technical Report. Ministère des Forêts, de la Faune et des Parcs, Gouvernement du Québec. Québec, Canada. URL: https://mrnf.gouv.qc.ca/documents/forets/connaissances/recherche/Guide-Artemis-Capsis.pdf. direction de la recherche forestière.

[41] Remsoft Inc., 2025. Woodstock [software]. URL: https://www.remsoft.com/products/woodstock/. accessed 2025-09-28.

[42] Smyth, C., Xu, Z., Lemprière, T., Kurz, W., 2020. Climate change mitigation in British Columbia's forest sector: GHG reductions, costs, and environmental impacts. Carbon Balance and Management 15, 1–22. doi:10.1186/s13021-020-00155-2.

[43] Spatial Planning Systems Inc., 2025. Patchworks [software]. URL: https://www.spatial.ca/patchworks/. accessed 2025-09-28.

[44] UBC FRESH Lab, 2024. Ws3 continuous integration workflow. https://github.com/UBC-FRESH/ws3/blob/main/.github/workflows/ci.yml. URL: https://github.com/UBC-FRESH/ws3/blob/main/.github/workflows/ci.yml. accessed 2025-09-28.

[45] UBC-FRESH Lab, 2025a. ecotrust-dss: WS3-backed decision support application. doi:10.5281/zenodo.17330007.

[46] UBC-FRESH Lab, 2025b. spades_ws3: Linking WS3 harvest scheduling with SpaDES [software]. doi:10.5281/zenodo.17330027.

[47] Yan, Y., 2024. msc_walter_yan: Carbon-aware harvest scheduling case study [software]. URL: https://github.com/UBC-FRESH/msc_walter_yan. gitHub repository (accessed 2025-09-28).

[48] Yan, Y., 2025. A Mathematical Modeling Framework to Optimize the Climate Change Mitigation Potential of the Forest Sector in British Columbia, Canada. Master's thesis. University of British Columbia. doi:10.14288/1.0449036.

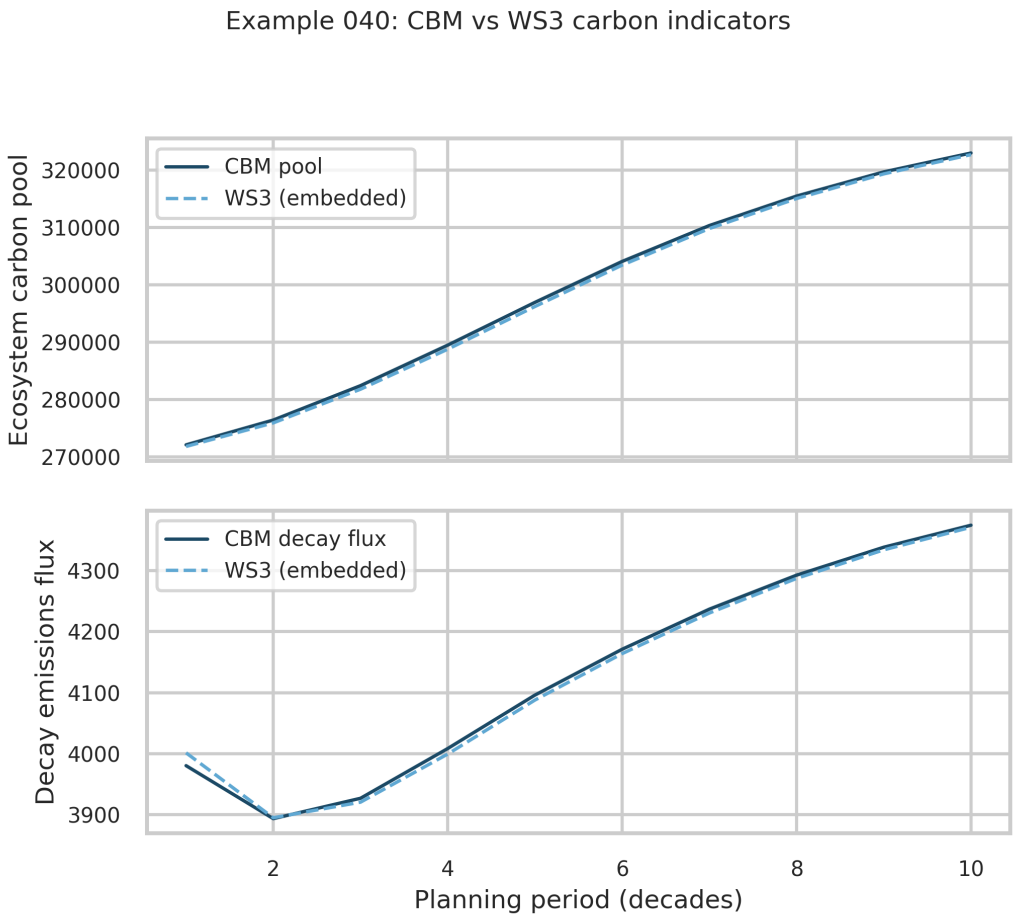


Figure 8: Neilson-hack demonstration (Example 040): comparison of libCBM (solid) and WS3-embedded (dashed) carbon indicators after bootstrapping pools/fluxes into WS3 as yield curves. Top: ecosystem carbon pools; bottom: aggregate decay emissions flux. In a no-disturbance scenario, the aggregated indicators match closely; with harvesting, flux gaps reflect known limitations of the Neilson hack due to CBM DOM spin-up assumptions mapping time to age (Figure 8).

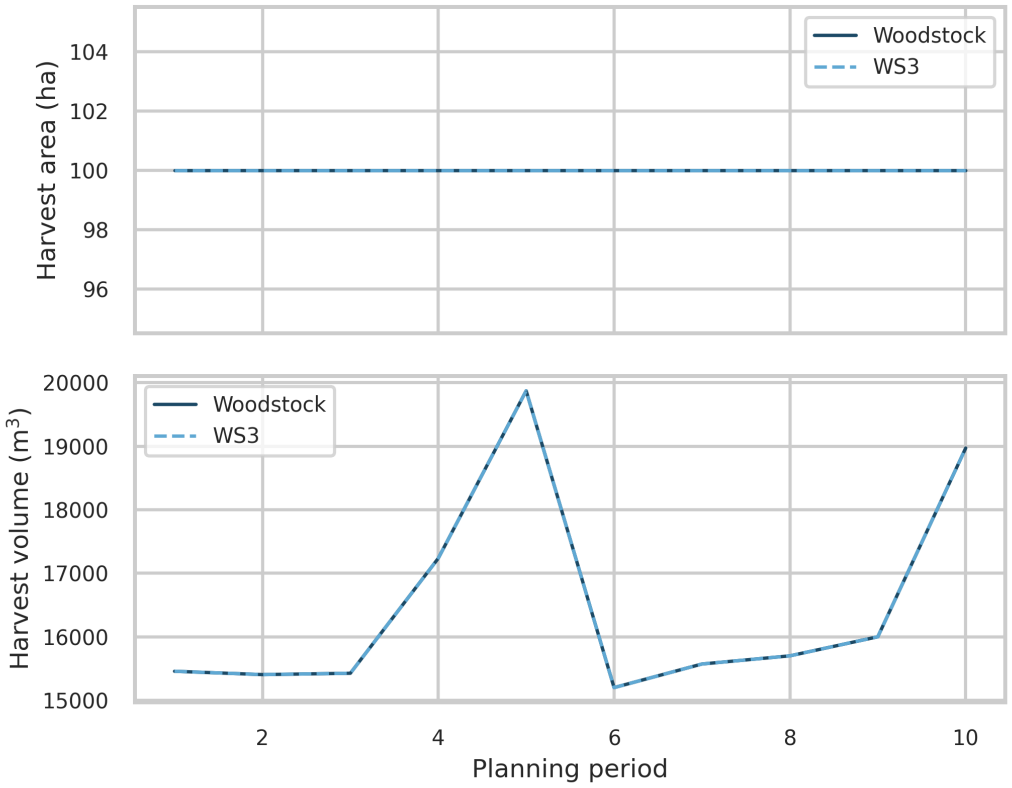


Figure 9: Period-wise parity of harvested area and harvested volume between WS3 and Woodstock on the toy dataset (identical schedule). Values are loaded from the accompanying CSV artifact.